DeepJoin: Joinable Table Discovery with Pre-trained Language Models

Yuyang Dong
NEC Corporation
dongyuyang@nec.com

Chuan Xiao
Osaka University
Nagoya University
chuanx@ist.osaka-u.ac.jp

Takuma Nozawa
NEC Corporation
nozawa-takuma@nec.com

Masafumi Enomoto
NEC Corporation
masafumi-enomoto@nec.com

Masafumi Oyamada
NEC Corporation
oyamada@nec.com

ABSTRACT

Due to the usefulness in data enrichment for data analysis tasks, joinable table discovery has become an important operation in data lake management. Existing approaches target equi-joins, the most common way of combining tables for creating a unified view, or semantic joins, which tolerate misspellings and different formats to deliver more join results. They are either exact solutions whose running time is linear in the sizes of query column and target table repository, or approximate solutions lacking precision. In this paper, we propose DeepJoin, a deep learning model for accurate and efficient joinable table discovery. Our solution is an embedding-based retrieval, which employs a pre-trained language model (PLM) and is designed as one framework serving both equi- and semantic (with a similarity condition on word embeddings) joins for textual attributes with fairly small cardinalities. We propose a set of contextualization options to transform column contents to a text sequence. The PLM reads the sequence and is fine-tuned to embed columns to vectors such that columns are expected to be joinable if they are close to each other in the vector space. Since the output of the PLM is fixed in length, the subsequent search procedure becomes independent of the column size. With a state-of-the-art approximate nearest neighbor search algorithm, the search time is sublinear in the repository size. To train the model, we devise the techniques for preparing training data as well as data augmentation. The experiments on real datasets demonstrate that by training on a small subset of a corpus, DeepJoin generalizes to large datasets and its precision consistently outperforms other approximate solutions'. DeepJoin is even more accurate than an exact solution to semantic joins when evaluated with labels from experts. Moreover, when equipped with a GPU, DeepJoin is up to two orders of magnitude faster than existing solutions.

PVLDB Reference Format:
Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. DeepJoin: Joinable Table Discovery with Pre-trained Language Models. PVLDB, 16(10): 2458 - 2470, 2023.
doi:10.14778/3603581.3603587

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 10 ISSN 2150-8097.
doi:10.14778/3603581.3603587

1 INTRODUCTION

Given a table repository and a query table with a specified join column, joinable table discovery finds the target tables that can be joined with the query. Due to the demonstrated usefulness in data enrichment [10, 16], joinable table discovery has become a key procedure in data lake management and serves various downstream applications, especially those involving data analysis.

For joinable table discovery, early attempts mainly targeted equi-joins [64, 66], which are the most common way of combining tables for creating a unified view [15] and can be easily implemented using SQL. To deliver more joins results for heterogeneous data, recent approaches [16, 17] studied semantic joins, which join on cells with similar meanings via word embedding, so as to handle data with misspellings and discrepancy in formats/terminologies (e.g., "American Indian & Alaska Native" v.s. "Mainland Indigenous"). There are two major limitations in these solutions. First, they only apply to a single join type. Second, most of them are exact algorithms with a worst-case time complexity linear in the product of query column size and table repository size, and thus their scalability is dubious. Despite the existence of an approximate algorithm for equi-joins [66], it is based on MinHash sketches [6] and has to convert the joinability condition to a Jaccard similarity condition, which is non-equivalent and introduces many false positives. Moreover, it is sometimes even slower than an exact algorithm [64].

Seeing the limitations of existing solutions, we propose DeepJoin, a deep learning model designed in a *two-birds-with-one-stone* fashion such that both equi- and semantic joins can be served with one framework. In particular, DeepJoin targets textual attributes with fairly small cardinalities that can fit with language models. To cope with semantic joins, it works on a similarity condition of word embeddings and finds similar textual columns as exactly as possible. To resolve the efficiency issue, it finds joinable tables via an **embedding-based retrieval**. In particular, we employ an embedding model to transform columns to a vector space. By metric learning, columns with high joinability are close to each other in the vector space. Then, to find the top-k target columns ranked by joinability, we resort to a state-of-the-art approximate nearest neighbor search (ANNS) algorithm [38], whose time complexity is *sublinear* in the table repository size.

DeepJoin utilizes a **pre-trained language model** (PLM) for column embedding. PLMs, such as BERT [14], have gained popularity in various data management tasks that involve natural language processing. A salient property of modern PLMs is that they are

transformer networks [55] featuring the attention mechanism, thus not only good at capturing the semantics of column contents for semantic joins, but also able to focus on the cells that are more probable to match in equi-joins, assuming that the query column has a similar distribution to those in the repository. As such, our model gains the capability of handling both join types, only needing the PLM to be fine-tuned on the data labeled for either equi- or semantic joins. In addition, PLMs produce a fixed-length vector, meaning that the following index lookup and search are *independent* of the column size, hence along with the ANNS, addressing the scalability issue in existing solutions. Since PLMs take a text sequence as input, by prompt engineering, we propose a set of options that contextualizes a column to a text sequence. To train the model in a self-supervised manner, we devise a series of techniques to effectively generate positive and negative examples. Moreover, our training features a data augmentation technique, through which our model can learn that the joinability is insensitive to the order of cells in a column.

We conduct experiments on two real datasets and evaluate DeepJoin equipped with two state-of-the-art PLMs. We show that by training on a small subset (30k columns) of the corpus, DeepJoin *generalizes* well to large datasets (1M columns). In particular, DeepJoin outperforms alternative approximate solutions in all the settings, and reports an average precision of 72% for equi-joins and 91% for semantic joins and an average NDCG of 81% for equi-joins and 75% for semantic joins. To test the effectiveness of semantic joins, we also evaluate DeepJoin using data labeled by our database researchers, and the results show that DeepJoin is even better by a margin of 0.105 – 0.165 F1 score than PEXESO [17], an exact solution we use to label DeepJoin's training data. An ablation study demonstrates the usefulness of the proposed contextualization and data augmentation techniques. For scalability test, we vary dataset size from 1M to 5M columns. Even if equipped with a CPU, DeepJoin exhibits superb scalability and is 7 – 57 times faster than existing solutions. With the help of a GPU, DeepJoin outperforms them by up to two orders of magnitude.

Contributions. (1) We propose DeepJoin, a framework for joinable search discovery in a data lake. Our solution targets targets textual attributes with fairly small cardinalities, and is able to detect both equi-joinable and semantically joinable tables. (2) We design the search in DeepJoin as an embedding-based retrieval which employs a fine-tuned PLM for column embedding and ANNS for fast retrieval. The search time complexity is sublinear in the table repository size, and except the column embedding, the search time is independent of the column size. (3) We propose a prompt engineering method to transform column contents to a text sequence fed to the PLM. (4) We devise techniques for training data preparation and data augmentation, as well as a metric learning method to fine-tune the PLM in a self-supervised manner. (5) We conduct experiments to show that our model generalizes well to large datasets and it is accurate and scalable in finding joinable tables.

Furthermore, we would like to mention the following facts: (1) For the embedding-based retrieval in our model, the results of ANNS are directly output as the results of joinable table discovery, whereas more advanced paradigms exist, such as two-stage retrieval [11] which finds a set of candidates by ANNS and ranks the candidates by a more sophisticated model. (2) DeepJoin is not limited to the two PLMs evaluated in our experiments, because the PLM can be regarded as a plug-in in our framework. As such, we expect the performance of DeepJoin can be further improved by using more advanced retrieval paradigms or PLMs.

2 PRELIMINARIES

2.1 Problem Definition

Given a data lake of tables, we extract all the columns in these tables, except those unlikely to appear in a join predicate (e.g., BLOBs), and create a repository of tables, denoted by $\mathcal{X}$. Given a query column Q, our task is to search $\mathcal{X}$ and find the columns joinable to Q. In this paper, we target equi-joins and semantic joins. Next, we define the joinability for these two types, respectively.

Given a query column Q and a target column X in $\mathcal{X}$, the joinability from Q to X is defined by the following equation.

$$jn(Q, X) = \frac{|Q_M|}{|Q|}, \tag{1}$$

where $|\cdot|$ measures the size (i.e., the number of cells) of a column, and Q_M is composed of the cells in Q that have at least one match in X. Here, the term "match" depends on the join type, i.e., an equi-join or a semantic join. We normalize the size of Q_M by the size of Q to return a value between 0 and 1. Moreover, the joinability is not always symmetric, depending on the definition of Q_M.

For equi-joins, we model each column as a set of cells by removing duplicate cell values, and define the equi-joinability as follows.

Definition 2.1 (Equi-Joinability). The equi-joinability from the query column Q to a target column X in $\mathcal{X}$ counts the intersection between Q and X, normalized by the size of Q; i.e., in Equation 1,

$$Q_M = Q \cap X. \tag{2}$$

The equi-joinability defined above counts the the distinct number of cells in Q that match those in X, and thus can be used to measure the equi-joinability [64]. Our method can be also extended to the case when columns are modeled as multisets, so as to support one-to-many, many-to-one, and many-to-many joins. In this case, we may measure the joinability by the number of join results and normalize it by the product of $|Q|$ and $|X|$, instead of $|Q|$ in Equation 1.

For semantic joins, we consider string columns and embed the value of each cell to a metric space $\mathcal{V}$ (e.g., word embedding by fastText [19]). As such, each string column is transformed to a multiset of vectors. We abuse the notation of a column to denote its multiset of vectors. Then, the notion of vector matching is defined as follows.

Definition 2.2 (Vector Matching). Given two vectors v_1 and v_2 in $\mathcal{V}$, a distance function d, and a threshold τ, v_1 matches v_2 if and only if the distance between v_1 and v_2 does not exceed τ. We use notation $M_\tau^d(v_1, v_2)$ to denote if v_1 matches v_2; i.e., $M_\tau^d(v_1, v_2) = 1$, iff. $d(v_1, v_2) \leq \tau$, or 0, otherwise.

Given a query column Q and a target column X, the semantic-joinability is defined using the number of matching vectors [1].

[1] Besides this definition, semantic joins are also investigated in [23], yet it studies the problem of performing joins rather than searching for joinable columns.

Definition 2.3 (Semantic-Joinability). The semantic-joinability from Q to X counts the number of vectors in Q having at least one matching vector in X, normalized by the size of Q; i.e., in Equation 1,

$$Q_M = \{\, q \mid q \in Q \wedge \exists x \in X \text{ s.t. } M_\tau^d(q,x) = 1 \,\}. \tag{3}$$

An advantage of the above definitions is that for both equi- and semantic-joinability, the training data can be labeled by an exact algorithm (e.g., JOSIE [64] and PEXESO [17]) rather than experts, so that our model can be trained in a self-supervised manner. Following the above definitions, we model the problem of joinable table discovery as the following top-k search problem, where joinability jn is defined using either Definition 2.1 or 2.3.

Problem 1 (Joinable Table Discovery). *Given a query column Q and a repository of target columns $\mathcal{X}$, the joinable table discovery problem is to find the top-k columns in $\mathcal{X}$ with the highest joinability from Q. Formally, we find a subset $\mathcal{R} \subseteq \mathcal{X}$, $|\mathcal{R}| = k$, and* $\min\{\, jn(Q,X) \mid X \in \mathcal{R} \,\} \geq jn(Q,Y), \forall Y \in \mathcal{X} \backslash \mathcal{R}$.

In this paper, we focus on dealing with textual columns. For numerical columns, a typical solution is utilizing the statistical feature vector in Sherlock [28], which converts a numerical column to a vector, and then we can invoke a vector search to look for joinable columns having similar statistics to the query column.

2.2 State-of-the-Art

JOSIE [64], a state-of-the-art solution to the equi-joinable table discovery problem, regards the problem as a top-k set similarity search with overlap $|Q \cap X|$ as similarity function, and builds its search algorithm upon prefix filter [7] and positional filter [59], which have been extensively used for solving set similarity queries. JOSIE creates an inverted index over $\mathcal{X}$, which regards each cell value as a token and maps each token to a postings list of columns having the token. Then, it finds a set of candidate columns by retrieving the postings lists for a subset of the tokens in Q (called prefix). Candidates are verified for joinability and the top-k is updated. While index access and candidate verification are processed in an alternate manner, JOSIE features the techniques to determine the their order, so as to optimize for long columns and large token universes, which are often observed in data lakes.

PEXESO [17] is an exact solution to semantic-joinable table discovery. It employs pivot-based filtering [8], which selects a set of vectors as pivots and pre-computes distances to these pivots for all the vecotrs in the columns of $\mathcal{X}$. Then, given the vectors of the query Q, non-matching vectors can be pruned by the triangle inequality. A hierarchical grid index is built to filter non-joinable columns when counting the number of matching vectors.

As for the weakness, JOSIE inherits the limitation of prefix filter, whose performance highly depends on the data distribution and yields a worst-case time complexity of $O(|\mathcal{X}| \cdot (|Q| + \overline{|X|}))$, where $\overline{|X|}$ stands for the average size of the columns in $\mathcal{X}$. For PEXESO, despite a claimed sublinear search time complexity of $O(\log |\mathcal{X}_V| \cdot \log |Q|)$, where $\mathcal{X}_V$ denotes the multiset of all the vectors in the repository, it relies on a user-specific threshold for the count of matching vectors. This does not apply to the top-k case, and the algorithm is downgraded to be linear in $|\mathcal{X}_V| \cdot |Q|$, because at the early stage of search, due to the low count of matching vectors in the temporary top-k results, the pruning power of the grid index is next to none.

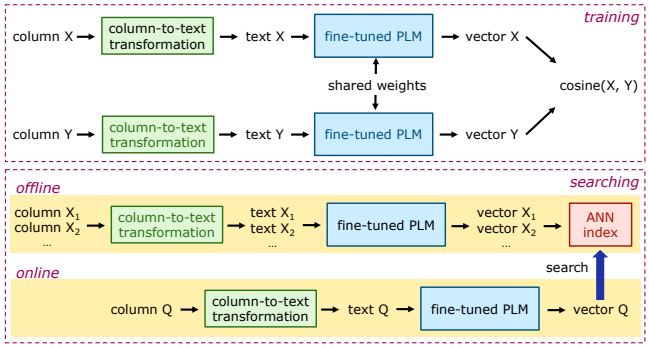


Figure 1: Overview of the DeepJoin model.

Table 1: Column-to-text transformation options.

Name	Pattern
col	$cell_1$, $cell_2$, . . ., $cell_n$
colname-col	$column_name$: col.
colname-col-context	$colname-col$: col. $table_context$
colname-stat-col	$column_name$ contains n values (max_len, min_len, avg_len): col
title-colname-col	$table_title$. $colname-col$
title-colname-col-context	$title-colname-col$. $table_context$
title-colname-stat-col	$table_title$. $colname-stat-col$.

In general, for search time, both JOSIE and PEXESO are linear in the product of column size and repository size, which compromises their scalability to long columns and large datasets. On the other hand, it is unnecessary to always find an exact answer, because data scientists are usually concerned with only part of the top-k results and will choose a subset of them for the join. For this reason, we will design our solution with the following two goals: (A) it returns an approximate answer with *sublinear* time in $|\mathcal{X}|$, and (B) by encoding the query column to a fixed length, it is *independent* of $|Q|$ and $\overline{|X|}$ during index lookup and search.

Apart from exact solutions, LSH Ensemble [66] is an approximate solution to equi-joinability. It partitions the repository and computes MinHash sketches [6] with parameters tuned for each partition. Unlike JOSIE, it targets the thresholded problem (i.e., $\frac{|Q \cap X|}{|Q|} \geq t$), which requires a user-specified threshold t and thus is less flexible than computing top-k. Although adaptation for the top-k problem is available, it suffers from low precision due to the many false positives introduced by transforming overlap similarity to Jaccard similarity for the use of MinHash, and it sometimes runs even slower than JOSIE [64].

3 THE DEEPJOIN MODEL

Figure 1 illustrates the overview of our DeepJoin model. In DeepJoin, the joinable table discovery is essentially an embedding-based retrieval, which has recently been adopted in various retrieval tasks, such as long text retrieval [33] and search in social networks [27]. An embedding-based retrieval usually employs metric learning or meta embedding search to learn embeddings for target data such that the semantics can be measured in the embedding space, especially when target data are highly sparse or the semantics is hard to define by an analytical model. Another key benefit is, by

SANTOS: Relationship-based Semantic Table Union Search

AAMOD KHATIWADA*, Northeastern University, USA
GRACE FAN*, Northeastern University, USA
ROEE SHRAGA, Northeastern University, USA
ZIXUAN CHEN, Northeastern University, USA
WOLFGANG GATTERBAUER, Northeastern University, USA
RENÉE J. MILLER, Northeastern University, USA
MIREK RIEDEWALD, Northeastern University, USA

Existing techniques for unionable table search define unionability using metadata (tables must have the same or similar schemas) or column-based metrics (for example, the values in a table should be drawn from the same domain). In this work, we introduce the use of semantic relationships between pairs of columns in a table to improve the accuracy of the union search. Consequently, we introduce a new notion of unionability that considers relationships between columns, together with the semantics of columns, in a principled way. To do so, we present two new methods to discover the semantic relationships between pairs of columns. The first uses an existing knowledge base (KB), and the second (which we call a "synthesized KB") uses knowledge from the data lake itself. We adopt an existing Table Union Search benchmark and present new (open) benchmarks that represent small and large real data lakes. We show that our new unionability search algorithm, called SANTOS, outperforms a state-of-the-art union search that uses a wide variety of column-based semantics, including word embeddings and regular expressions. We show empirically that our synthesized KB improves the accuracy of union search by representing relationship semantics that may not be contained in an available KB. This result hints at a promising future of creating synthesized KBs from data lakes with limited KB coverage and using them for union search.

CCS Concepts: • **Information systems → Information integration**.

Additional Key Words and Phrases: Data lakes, Table Discovery, Knowledge Graphs

ACM Reference Format:
Aamod Khatiwada*, Grace Fan*, Roee Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J. Miller, and Mirek Riedewald. 2023. SANTOS: Relationship-based Semantic Table Union Search. *Proc. ACM Manag. Data* 1, 1, Article 9 (May 2023), 25 pages. https://doi.org/10.1145/3588689

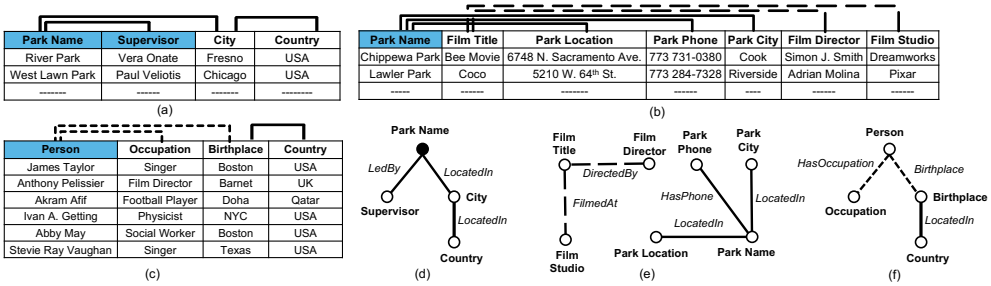
1 INTRODUCTION

Table search is of growing practical relevance [7, 23, 30, 49]. It allows data scientists to find datasets needed for analysis or training of machine learning algorithms [14, 33]. Google's dataset search, for example, uses keyword search over metadata which Google incentivizes data owners to use in

* Khatiwada and Fan are lead authors.

Authors' addresses: Aamod Khatiwada*, Northeastern University, USA, khatiwada.a@northeastern.edu; Grace Fan*, Northeastern University, USA, fan.gr@northeastern.edu; Roee Shraga, Northeastern University, USA, r.shraga@northeastern.edu; Zixuan Chen, Northeastern University, USA, chen.zixu@northeastern.edu; Wolfgang Gatterbauer, Northeastern University, USA, w.gatterbauer@northeastern.edu; Renée J. Miller, Northeastern University, USA, miller@northeastern.edu; Mirek Riedewald, Northeastern University, USA, m.riedewald@northeastern.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART9 $15.00
https://doi.org/10.1145/3588689



Park Name	Supervisor	City	Country
River Park	Vera Onate	Fresno	USA
West Lawn Park	Paul Veliotis	Chicago	USA
-------	------	-------	--------

(a)

Park Name	Film Title	Park Location	Park Phone	Park City	Film Director	Film Studio
Chippewa Park	Bee Movie	6748 N. Sacramento Ave.	773 731-0380	Cook	Simon J. Smith	Dreamworks
Lawler Park	Coco	5210 W. 64th St.	773 284-7328	Riverside	Adrian Molina	Pixar
-----	------	-------	------	----	------	------

(b)

Person	Occupation	Birthplace	Country
James Taylor	Singer	Boston	USA
Anthony Pelissier	Film Director	Barnet	UK
Akram Afif	Football Player	Doha	Qatar
Ivan A. Getting	Physicist	NYC	USA
Abby May	Social Worker	Boston	USA
Stevie Ray Vaughan	Singer	Texas	USA

(c)

Fig. 1. (a) A table about parks. Relationships between `Park Name` and `Supervisor` (`ledBy`) and between `Park Name` and `City` (`locatedIn`) are found and depicted by the solid lines above the table. We have also found a relationship between `Country` and `City` (`locatedIn`). (b) A table about parks and films shown in the park. Some of the relationships we found are shown by solid and dashed lines above the table. (c) A table about famous people. (d) A semantic graph for Table (a) with the relationships found related to the root `Park Name`. (e), (f) Semantic graphs for Tables (b), (c), respectively.

a standard way [4]. However, when dealing with enterprise data or open data, keyword search via textual queries or over metadata is challenging because metadata may be missing, inconsistent, or incomplete [1, 10, 32, 47]. Hence, table search for those domains must rely on the data available within given tables instead of relying on well-curated metadata [7, 23].

In this work, we focus on *table union search*, which is the problem of finding tables that can union with a query table and possibly extend it with more rows. Prior work by Nargesian et al. [33] considers two tables to be *unionable* if "they share attributes from the same domain". We believe that this is a necessary, but not sufficient condition. In particular, we argue that the relationships modeled by pairs of attributes in a table should share a common semantic meaning. We illustrate this with an example.

Example 1. *Consider Tables (a), (b) and (c) in Fig. 1 describing (a) parks, (b) films shown in different parks, and (c) the birth places of famous people, respectively. Suppose the user supplies Query Table (a) and Tables (b), (c) reside in a data lake. The user wants to find data lake tables that union with the Query Table (a). Typically, union search has been defined to permit either the full query table or a projection of it to union with the full data lake table or a projection of it [3]. Now, if we only consider attribute unionability, Table (c) may be considered the most unionable table with three unionable attributes (*`Person`, `Birthplace`, `Country`*) having high attribute unionability scores with the attributes (*`Supervisor`, `City` *and* `Country`*) in the query table. This may be true if we are considering data values [33] or if we consider the attribute names and values [3, 21, 24]. However, if we consider the relationships in the tables, then we would notice that in Query Table (a),* `City` *is where the* `Park` *is located (and the person in* `Supervisor` *works), but in Table (c),* `Birthplace` *contains cities where the people in the* `Person` *column were born. Deciding unionability based only on attribute unionability, without considering the semantics of the relationships between attributes, can lead to false positives. Even worse, it can lead to the omission of good answers like Table (b) which has information about parks and the city in which they are located, but might be considered less unionable than (c) based solely on column semantics.*

We present a new definition of unionability based on both the attribute semantics and the *relationship semantics between attributes*. Relationships have been used to a limited extent in web table search. Their tables are often assumed to be "entity tables" (i.e., all attributes are properties of a known common entity represented by a single attribute) [7, 8, 28] or they use a subject column and a single relationship from that column to search for tables [43]. However, we do not assume

that every table has a single subject attribute that all other attributes describe, nor that we can automatically detect such an attribute. While the entity-table assumption may be mostly true in web tables, data lake tables (in both open data and enterprise lakes) tend to be much wider and may describe the interplay between more than one entity.

In this work, we provide a new definition of table union search. Given a query table Q and a set of data lake tables $\mathcal{T}$, the top-k table union search problem is to find the best k tables from $\mathcal{T}$ that can be unioned with the query table by taking the column and relationship semantics of all involved tables into account.

Example 2. *The columns* `City` *and* `Country` *in Table (a) are unionable with* `Birthplace` *and* `Country` *in Table (c) because the columns are unionable and their relationships are the same (*`locatedIn`*). If a user is interested in amassing a collection of cities and their properties like location, then these tables are unionable. However, if a user wants to collect a set of park supervisors and their properties (the city and country in which they work), then Tables (a) and (c) are not unionable. This is because the relationship between* `Supervisor` *and* `City` *(*`worksIn`*) is not the same as the relationship between* `Person` *and* `Birthplace` *(*`bornIn`*). A user may consider Table (b) to be unionable with Table (a), but only on* π*(*`Park Name, City, Country`*). If we only consider attribute semantics, we might mistakenly say* `Park Name, Supervisor, City` *is unionable with* `Park Name, Film Director, Park City`*. But the relationship semantics reveal that* `Supervisor` *has a different relationship to* `Park Name` *than* `Film Director` *has to* `Park Name`*.*

1.1 Contributions

Our contributions can be summarized as follows.

• *Relationship-based Semantic Table Union Search.* We present a new definition of table union search that comprehensively integrates column and relationship semantics.

• *KB Solution.* We present a solution to the relationship-based semantic table union search problem that uses a knowledge base (KB) to determine column and relationship semantics. Our solution creates semantic graphs for tables whose nodes are columns and edges are binary relationships found in the KB (depicted in Fig. 1(d)-(f) and explained in Sec. 3). We present a scoring function to match the semantic graph of a data lake table to that of a query table.

• *Synthesized KB Solution.* We present a second, novel solution to the search problem that uses the data lake itself to determine if a set of columns have the same or similar semantics. Our solution determines if the binary relationship between two columns has similar semantics with other binary relationships in the data lake. We show how this *synthesized KB* can be used to create semantic graphs and solve the table union search problem.

• *Empirical Evaluation.* Our two solutions were designed to be used in concert with the Synthesized KB compensating for a curated KB that has only partial coverage over a data lake (which is typical when using both open KBs and proprietary enterprise KBs). We show that when used together, our solutions outperform a SOTA table union baseline. We also experiment with using each solution independently. Finally, we use the Synthesized KB together with different sized samples of the curated KB to understand how accurate our solution is as the coverage of the curated KB decreases.

• *Benchmarks.* We develop (and openly share) three new relationship union benchmarks. In the first, we reuse part of the TUS benchmark [33] and label it with relationship semantics. In the second, we use real open data tables and queries, and label a groundtruth. The third is a larger data lake with open data tables and queries, where we only label the query results that either our technique or a baseline technique returns. Hence, this larger benchmark can be used to evaluate precision and efficiency, but not recall.



Searching Data Lakes for Nested and Joined Data

Yi Zhang*
AWS AI Labs
imyi@amazon.com

Peter Baile Chen*
MIT
peterbc@mit.edu

Zachary G. Ives
University of Pennsylvania
zives@cis.upenn.edu

ABSTRACT

Exploratory data science is driving new platforms that assist data scientists with everyday tasks, such as integration and wrangling, to assemble training datasets. Such tools take scientists' work-in-progress data as a *search object* (table or JSON) and find relevant supplementary data from an organizational *data lake*, which can be unioned or joined with the current data. Existing data lake search tools find *single*, relational tables to match or join with a search object. Yet many data science applications revolve around hierarchical data, which can only be matched by creating views that simultaneously *join and transform several* tables in the data lake. In this paper, we extend the Juneau data lake search system [46] for this broader class of matches *at scale*. Our contribution is a *general* framework for efficiently merging ranked results to match hierarchical data, leveraging novel techniques for indexing and sketching, and incorporating existing single-table search techniques and ranking functions. We experimentally validate our methods' benefits and broad applicability using real data from data science computational notebooks. Our results indicate that, with different ranking functions, our approach can return the optimal set of views up to 4.8x faster and 43% more related compared to heuristics, and increase the data domain coverage by up to 28%. In a case study to show the utility of our results to data science downstream tasks, we reduce regression error by up to 6.6%, and improve classification accuracy by up to 19.5%.

PVLDB Reference Format:
Yi Zhang, Peter Baile Chen, and Zachary G. Ives. Searching Data Lakes for Nested and Joined Data. PVLDB, 17(11): 3346 - 3359, 2024.
doi:10.14778/3681954.3682005

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/juneau-project/juneau.

1 INTRODUCTION

Data scientists are increasingly using tools that assist with data exploration [19], machine learning model training, and workflow definition tasks. These scientists search over tables and code from public repositories such as GitHub [44], as well as enterprise or public data lakes [34, 46–48] to find supplemental data that can augment their working dataset(s). In the same spirit as Code Llama [39] or GitHub CoPilot [18] for software engineering, such tools assist programmers with data engineering tasks such as discovery and data augmentation [34, 46], as well as data wrangling and cleaning steps [6, 46]. Such tools complement large language models like GPT, Claude, and Gemini. Unlike LLMs that make suggestions based on their parametric knowledge, search tools do not "hallucinate" results. Moreover, they can incorporate policies for domain-specific data types (e.g., gene sequence similarity) or for preferring one source over another (e.g., based on freshness).

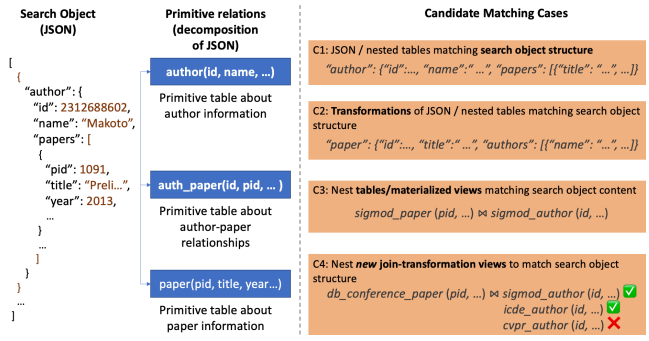


Figure 1: Schema-compliant matches to a JSON search object may come directly or through joins followed by nesting transformations.

The core of such tools is a *search system over data lakes* [13, 32, 34]: Here, the user invokes a query via a *search object* (in existing systems, a table) and the system queries data lake resources for (1) union-compatible tables [13, 33, 46, 48] that can extend training or test datasets with new tuples, (2) joinable tables [16, 46, 47] that integrate data or add machine learning features, or (3) common computational steps found as used in prior workflows [22, 24, 44, 46]. In contrast to web table search [5, 37], the critical challenges of developing such tools lie in understanding the (semantic) compatibility of the schema and detecting the primary-key overlap with the query table [13, 46] . There are two broad classes of existing systems: union-compatible search strives to find tables that *match the search object's schema* while providing novel rows; join-compatible search tends to be non-goal-directed, opportunistically finding tables that appear to join with the existing search table. Recent work [26] shows how to use *full disjunction* to knit the results together to form one output schema.

However, the paradigm of single-table matches to a search object is overly restrictive. Data scientists frequently work with JSON files, nested Dataframes, and composite tabular results (views) that join, pivot, and explode imported tables. Here, to provide the best search results — *the search system must combine subsets of indexed data into new hierarchical or joined data matching the search object structure.* New combinations of indexed tables (i.e., automatically discovered views) better leverage existing data and provide additional insights.

*This work was done while the authors were at the University of Pennsylvania.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 11 ISSN 2150-8097.
doi:10.14778/3681954.3682005

This requires that the data lake system can *not only find 1NF tables* but *explore ways of creating query transformations over combinations of indexed tables to match the query.*

Example 1.1. Snapshots of the popular DBLP publications database (dblp.org) have been exported for use in data science tasks, e.g., in notebooks shared on kaggle.com. These snapshots are often subsets of the entire database, taken at different points in time, and their representation may range from flat CSVs[1] to nested JSON[2]. A data lake could also have complementary publication data (e.g., venues not indexed in DBLP) extracted from conference pages or the ACM Digital Library. Now (see Figure 1), if a data scientist takes a JSON snapshot of DBLP and searches for similar data to train a classifier that recognizes CS papers, we would like to return *all* relevant author-publication data (converted to the appropriate structure), including (C1) objects that directly match the nested schema, authors with nested papers; (C2) "pivoted" JSON, in a different structure with semantically compatible content (e.g., papers with nested authors); (C3) nesting of flat tables or views representing the same concepts as the JSON; (C4) nesting of join queries over 1NF relations, matching the JSON schema.

In a similar fashion, a data scientist may take portions of the above data and create an intermediate table for machine learning training, which joins data from different levels of the DBLP hierarchy. At this point, they may want to find matches, which, while "flat," can be matched by joining 1NF data (which, in turn, could have been gathered by decomposing other JSON structures). Unlike prior join-compatible search approaches, this task is *goal-directed* in terms of what joins; unlike prior union-compatible search approaches, this task requires more than single-table matches.

Conventional table search within a data lake introduces two key challenges: identifying semantic and value overlap across thousands to millions of tables, which has generally been addressed using sketch and embedding techniques [16, 45, 47]; and efficiently ranking the results, especially if there are multiple ranking measures [46]. Extending these ideas to heterogeneous and hierarchical *compositions* of tables and JSON/XML results in three additional critical challenges:

(1) Matches to the search object may be *partial*, requiring joins to produce a complete search result. In our example, the JSON can be normalized into three "primitive" relations: *author*, *paper*, *auth_paper*. Individual data objects might contain the information from all these (C1-3 in Figure 1); or might require us to find *other* tables to join, to form a complete match (C4). Efficiently returning the top-k results, when each result has an irregular number of joins, explodes the search space.
(2) Simply querying the primitive relations in top-k order, then joining them, does not produce the top-k answers: intuitively, high-scoring matches to papers may not actually join with high-scoring matches to authors (as per database papers with CVPR authors in Figure 1.) Therefore, the ranking function should measure the compatibility of a composition of primitive relations, and the score should be independent of the order of evaluation.
(3) Automatically and efficiently identifying which relations should join (without introducing large numbers of false positives) is nontrivial – this is aided tremendously by identifying fields that are good keys and also highly indicative of semantic content, e.g., DOIs, SSNs, etc.

[1]https://www.kaggle.com/jakboss/chunk-of-dblp-dataset
[2]https://www.kaggle.com/mathurinache/citation-network-dataset

Searching data lakes for nested and joined data requires novel indexing and pruning strategies. Broadly, as in Figure 1, the problem requires that we decompose the nesting structure of our hierarchical data and find matches to each nesting level (columns in 1NF tables, or in decompositions of other nested tables); then we return a ranked set of queries (views) over these matches as the answers. Note that among the search cases we have discussed, C4 is particularly challenging, as queries need to be created first. To address these problems, we must innovate in how we index content in tables, and perform top-k computations, with the assistance of the indices, across multiple tables while joining them.

A significant body of empirical work has been developed on finding the best features for ranking individual tables considering schema overlap, table overlap, and joinability (see Section 2). In this paper, we develop a general query system that can "plug in" existing individual table ranking strategies (with different degrees of benefits depending on the specific setup), and our focus is on scaling up running times of returning the (new) views over these tables. To facilitate such a query system, we address the following challenges.

Enumerating plans to join and nest data decomposed into 1NF, for a broad class of ranking functions. We *compose* partial matches (1NF primitive tables) from our data lake using nesting and join operations to match search object structure — favoring joins of results that originally came from the same sources. However, our system also considers cross-source joins, which are critical to applications in Linked Open Data and scientific settings with private data referencing entries in public databases.

Incremental maintenance of sketches and profiles. Existing data lake search systems developed *in-memory* sketches and data profiles. To support data compositions in millions of tables, these must be persisted and incrementally maintained.

Automatic index selection. To scale up the exploration of potential joins to produce a match, we develop solutions to determine combinations of fields to index across the data lake, in the form of automatic *data profile* selection.

To sum up, this paper makes the following contributions:

- A new class of data lake search tools over joined and nested data, whose task is to generate unions of nest-join queries over data lake tables to match a search object, with *customizable ranking functions or modules.*
- Identification of key requirements for ranking functions over candidate *views (over multiple tables)*, enabling an efficient *correlated top-k matching algorithm* to rank them.
- *Automatic* selection techniques for *profiling and sketching* millions of tables.
- Experimental validation of the viability and the generality of our methods, over a diverse benchmark suite of real data and notebooks, using state-of-the-art table scoring functions.

2 PRIOR WORK

Semantics of Table Search. The problem of searching a data lake for related tables has generated a significant interest [34].

Value-based table search (keyword search over tables). The earliest table search systems look for matches to words or values *within* tables. They rank table matches by how well the terms represent the tables. This line of work predates the data lake, and ranks tables by TF/IDF-like measures; tables are matched without considering the compatibility between their schemas. Such tools were largely targeted at enterprise networks [3, 15, 16, 20, 21, 43] and/or tables that appear on the web [4, 5, 37].

Unionable table search (table augmentation). Given a search table, these systems use schema alignment techniques to find similar columns. They rank tables based on schema similarity and how many new rows are added [48]. At scale, columns are usually compared using sketch-based techniques including LSH [16, 34, 46], as well as techniques borrowed from schema matching [27] and column or table embeddings [8, 10, 14] with Approximate Nearest Neighbor Search (ANNS). D^3L [2] further combined all these techniques. Beyond those ideas, SANTOS [25] recently proposed to use a knowledge base to discover the semantic relationships between pairs of columns in tables to find unionable tables. There are also efforts considering how data from data science computational notebooks can be leveraged as a data lakes [46], how computation over that data might be reorganized [23], and even how schema constraints might be inferred [41].

Joinable table search. Given a search table, this class of system identifies columns that can be joined to other columns from tables in the data lake, and ranks tables based on the estimated join compatibility [7, 17, 46, 47]. This line of work focuses on tables with matching join columns, but otherwise is not generally directed by a target schema. Similar to union-compatible table search, it is intractable to do pairwise comparisons of columns to find joinable pairs of columns, and therefore LSH and other sketch-based techniques are often used. Recently, DeepJoin [11] proposed an embedding-based retrieval method, which employs a pre-trained language model fine-tuned on equi- or semantic join data for column embedding and uses ANNS, i.e., HSNW [31] for fast retrieval.

The ALITE system [26] explores how to merge and align different (tabular) search results (as a query post-processing), using Full Disjunction as a means of null-padding tables as necessary to achieve union compatibility among the matches. However, since the returned matches are only constructed with tables returned by the query for individual tables, the search space is limited.

Prior efforts match an *individual* table against other "raw" tables. Our work addresses union-compatible *hierarchical* data and investigates techniques for scalably exploring joining and nesting *queries* over primitive tables.

Foundations of our work. As the foundations of our work, we extend an open-source, data lake search system, Juneau [46] [3], which encompasses many of the basic techniques described above. More specifically, the open-source version of Juneau provides three core capabilities, which are integrated into the Jupyter Notebook data science environment.

[3] https://github.com/juneau-project/juneau

Searching: Given a request from the user with a search table S, Juneau searches for other tables that can be combined with S by unioning or joining. It compares S against other tables using a series of "pluggable" metrics, including schema overlap, value overlap, and more. These are combined in a weighted linear fashion, and new metrics can be added, so long as the final function is monotonic with regards to the individual metrics. Juneau works as a general engine for *top-k query processing*, which ranks tables via this similarity score. Juneau's extensibility differentiates it from alternatives in the previous section. Compared to those single-similarity-metric systems [47], Juneau provides significant flexibility and customizability, but at the cost of more expensive search. For our problem which involves multiple matching criteria, this customizability is essential as well — but Juneau's algorithms are not scalable enough.

Indexing and index selection: To scale its search and top-k ranking algorithms to tens of thousands of tables, Juneau relies on additional strategies for indexing. A key capability is *data profile*, a means of identifying and indexing subsets of columns in each table belonging to a particular domain, which often co-occur. Intuitively, a data profile captures the **domain** that identifies semantically meaningful columns (e.g., a country) or combinations of columns (e.g., a country + continent). However, Juneau requires a significant extension to handle millions of tables, as we target.

Loading: As Juneau encounters tables (as they are created or loaded in the Jupyter Notebook environment), it automatically loads these into the data lake, in the process also extending any existing data profiles to consider the data in each new table.

The Juneau system provides a foundation and platform implementation for exploring query-based data lake search — but we also leverage other ideas from the literature.

A key aspect of table matching is the use of sketches for computing column compatibility (both for schema alignment and record linking): LSH ensembles allow us to find overlapping strings [48]; Kolmogorov-Smirnov sketches allow us to compare value distributions for numeric fields [42]. Our platform indexes and stores hierarchical data, including XML and JSON. To enable maximum flexibility during search (where the search object's hierarchy may not exactly match the stored object's hierarchy), we normalize all hierarchical data into 1NF relations, but maintain information about the original structure, as well as the foreign keys required to recompose them into original or pivoted hierarchies. To do this, we adopt ideas developed for storing XML in relations [9, 40]. For XML, this occurs by consulting the XML schema and creating or reusing relations (with appropriate schema elements) at $1:n$ nesting boundaries; then, on-the-fly as the XML is parsed, different components of the XML tree are directly inserted into the appropriate tables. Finally, our work on top-k query processing adapts ideas from Fagin's Threshold Algorithm [12] as well as the J^* algorithm [35] to iteratively explore options.

3 ENUMERATING POSSIBLE QUERIES

Our work explores how to create and rank *join-nest queries* over combinations of tables in a data lake, which can be unioned with a (hierarchical, i.e., non-1NF) search object. Our goal is a general architecture that supports many potential scoring functions (modules) from the literature. In this section, we identify (1) the potential

Table Overlap Estimation through Graph Embeddings

FRANCESCO PUGNALONI, Hasso Plattner Institute, University of Potsdam, Germany
LUCA ZECCHINI, University of Modena and Reggio Emilia, Italy
MATTEO PAGANELLI, University of Modena and Reggio Emilia, Italy
MATTEO LISSANDRINI, University of Verona, Italy
FELIX NAUMANN, Hasso Plattner Institute, University of Potsdam, Germany
GIOVANNI SIMONINI, University of Modena and Reggio Emilia, Italy

Discovering duplicate or high-overlapping tables in table collections is a crucial task for eliminating redundant information, detecting inconsistencies in the evolution of a table across its multiple versions produced over time, and identifying related tables. Candidate duplicate or related tables to support this task can be identified via the estimation of the largest table overlap. Unfortunately, current solutions for finding it present serious scalability issues for heavy workloads: Sloth, the state-of-the-art framework for its estimation, requires more than three days of machine time for computing 100k table overlaps.

In this paper, we introduce Armadillo, an approach based on graph neural networks that learns table embeddings whose cosine similarity approximates the *overlap ratio* between tables, i.e., the ratio between the area of their largest table overlap and the area of the smaller table in the pair. We also introduce two new annotated datasets based on GitTables and a Wikipedia table corpus containing 1.32 million table pairs overall labeled with their overlap. Evaluating the performance of Armadillo on these datasets, we observed that it is able to calculate overlaps between pairs of tables several times faster than the state-of-the-art method while maintaining a good quality in approximating the exact result.

CCS Concepts: • **Information systems** → *Information retrieval*; *Information integration*; *Deduplication*.

Additional Key Words and Phrases: table representation learning, graph neural networks, data lakes, web tables, table overlap, table matching

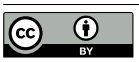
ACM Reference Format:
Francesco Pugnaloni, Luca Zecchini, Matteo Paganelli, Matteo Lissandrini, Felix Naumann, and Giovanni Simonini. 2025. Table Overlap Estimation through Graph Embeddings. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 228 (June 2025), 25 pages. https://doi.org/10.1145/3725365

1 Estimating table overlap

Tables are one of the most common formats used in data lakes and Web sources for organizing information that refers to a common schema [39]. For instance, a large amount of tabular data can be found in GitHub, which contains more than 10 million tables stored as CSV files [25], and Wikipedia, where more than 3 million HTML tables have been created throughout its history [2, 5].

In these large table corpora, it is frequent to find pairs of tables that share the same content across multiple cells, i.e., *overlapping* tables. This overlap, depending on its size and shape, can be a signal of table relatedness. For instance, a *vertical* overlap spanning several rows across a

Authors' Contact Information: Francesco Pugnaloni, Hasso Plattner Institute, University of Potsdam, Germany, francesco.pugnaloni@hpi.de; Luca Zecchini, University of Modena and Reggio Emilia, Italy, luca.zecchini@unimore.it; Matteo Paganelli, University of Modena and Reggio Emilia, Italy, matteo.paganelli@unimore.it; Matteo Lissandrini, University of Verona, Italy, matteo.lissandrini@univr.it; Felix Naumann, Hasso Plattner Institute, University of Potsdam, Germany, felix.naumann@hpi.de; Giovanni Simonini, University of Modena and Reggio Emilia, Italy, giovanni.simonini@unimore.it.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/6-ART228
https://doi.org/10.1145/3725365

R	Monument	City	Country	Year
r_1	TV Tower	Berlin	Germany	1965-1969
r_2	Big Ben	London	England	1843-1869
r_3	St. Davids	-	England	1915-1921
r_4	Colosseum	Rome	Italy	70-80
r_5	AquaClaudia	Rome	Unknown	52
r_6	Well of Life	Zagreb	Croatia	Unknown

S	Nation	Population	City
s_1	England	8,866,180	London
s_2	Italy	2 752 908	Rome
s_3	England	1 348	St.Davids
s_4	Unknown	52	Hum
s_5	Croatia	Unknown	Rastoke
s_6	Germany	3.755.251	Berlin

Fig. 1. Pair of tables that overlap in different ways. ($\langle$"*Unknown*", "52"$\rangle$, $\langle$"*Croatia*", "*Unknown*"$\rangle$) has area four and is the largest overlap obtainable without reordering rows and columns, while ($\langle$"*Berlin*", "*Germany*"$\rangle$, $\langle$"*Rome*", "*Italy*"$\rangle$, $\langle$"*London*", "*England*"$\rangle$) is the real largest overlap, it has area six and is obtained through a permutation of rows and columns. The overlap ratio is 6/18 (largest overlap area / area of the smaller table).

few columns may denote a pair of joinable tables. Similarly, a *horizontal* overlap can be a sign of unionability. Further, an overlap covering many rows and columns might represent a case of duplicate tables, table containment, or different versions of the same table [3].

Detecting overlaps is a challenging task for traditional set-based methods for data discovery, as it requires considering the alignment of cell values in a table and the structure of the tables themselves. To fill this gap, the concept of *largest overlap*, i.e., the largest common *rectangular* subtable existing between two tables under any row and column reordering, was recently introduced [54]. We present an illustrative example of largest overlap in Figure 1.

Identifying tables for which the largest overlap exceeds a certain area and/or presents a specific shape finds application in multiple real-world scenarios, from the detection of largely overlapping tables in a data lake, e.g., for subsequent operations of change propagation [3] or elimination of redundant information, to more specific tasks, e.g., the detection of copying dependencies across multiple sources [35] or the automated discovery of multi-column joins [54]. In addition, table overlap not only serves user-facing applications, but can also be used as a filtering or blocking method for downstream tasks, such as table reclamation [16].

In practice, all proposed use cases fall under two main scenarios: *(i) table matching*, i.e., retrieving all the clusters of related tables in a corpus, and *(ii) table querying*, i.e., given a *query table*, identifying a set of related tables in a corpus by building a ranking or using a similarity threshold. Since the first scenario requires comparing all possible pairs, it has a quadratic complexity, and even when working with datasets of a few thousand elements, a large number of comparisons needs to be performed. The second, instead, is linear with respect to the number of tables in the corpus. However, it can be equally challenging if computed in a brute-force manner, indeed Sloth [54], the only prior approach available for estimating the largest table overlap, takes more than two hours of machine time to compare 1 table with a data lake of 10K tables. Thus, in this paper, we analyze the problem of efficiently approximating the *maximum normalized table overlap*, referring to it as *overlap ratio*.

1.1 Table overlap ratio

We define the overlap ratio between two tables by extending the definition of the largest table overlap [54]. The intuition is to identify the sub-tables of maximum area, in terms of number of cells, that are shared between the two tables and normalize their area with respect to the area of the smaller table.

Given two tables R and S defined on the schemas X and Y respectively, we first define an attribute mapping M as the bijective function that links a subset of columns of the first table $X_M \subseteq X$ to another set $Y_M \subseteq Y$ of the second table. Each mapping determines a table overlap O_M, which corresponds

to the sub-table obtained from the intersection under bag semantics of the tuples in the columns linked by the mapping. We call the overlap area for the mapping M the table area of O_M, i.e., the number of cells contained in the table overlap generated by the mapping M. Among all the possible mappings, we are interested in identifying the ones with the maximum overlap area.

Example 1 (Largest table overlap). *Given the tables R and S in Figure 1 it is possible to identify several attribute mappings. Just to list a few, the mapping* M_1:⟨*Year*⟩→⟨*Population*⟩ *generates an overlap area of two, i.e., only two cells are shared between the two columns. The mapping* M_2:⟨*City, Country*⟩→⟨*City, Nation*⟩ *instead, represents a mapping of area six and since there are no other mappings that generate a larger area, it is a largest table overlap.*

Formally, we call $O(R, S)$ the set of table overlaps determined by all possible attribute mappings between R and S. We instead identify with $O^*(R, S) \subseteq O(R, S)$ the set of table overlaps with the largest overlap area, and their respective areas with $A^*(R, S)$.

Definition 1.1 (Overlap ratio). Let R, Y be tables defined on the schemas X, Y, A_R and A_S their table areas, and $A^*(R, S)$ their largest overlap area. We call

$$\theta(R, S) = \frac{A^*(R, S)}{min(A_R, A_S)} \in [0, 1]$$

overlap ratio between the tables R and Y.

Example 2 (Overlap ratio). *Table R and S have an area of* 6×4 *and* 6×3 *respectively, and their largest table overlap, deriving from* M_2*, is 6. Their overlap ratio is* $\theta(R, S) = \frac{6}{min(6\times4, 6\times3)} = \frac{6}{18} = 0.33$.

1.2 Efficient overlap ratio approximation

Although Sloth [54] provides a method to compute the largest overlap between pairs of tables, it has serious scalability issues. A possible approach to solving this problem is to consider methods that further approximate this calculation, such as considering tables as flattened sets of values and adapting a set-wise similarity measure for this purpose. The *Jaccard Similarity* [27] is generally used to compare textual data and defines the similarity of sets of elements as the ratio between the cardinality of their intersection and their union. It can be used to approximate the overlap ratio by finding the intersection under set semantics of values in a pair of tables and dividing their cardinality by the cardinality of their union. Since this approach uses the set semantics, it does not take into account the repetition of values in the tables. A solution to the problem is computing the Jaccard similarity under bag semantics, i.e., considering tables as bags of values that allow repetitions and dividing the cardinality of the bag intersection by the cardinality of the bag union. Since the number of values in the bag intersection is at most equal to the sum of the sizes of the two bags, it can predict at most a value of 0.5, hence, it cannot be used as it is. Two solutions are (1) scaling its result to the right interval by multiplying its prediction by two or (2) dividing the cardinality of the bag intersection by the size of the smaller bag.

Nonetheless, none of these approaches consider the structure of the tables and thus they fail when the table pairs have more than one common sub-table. In particular, the last method assumes that all values in the bag intersection are part of the same largest common subtable, always predicting an overlap greater or equal to the real one.

Example 3 (Approximate overlap ratio). *For the tables in Figure 1, the Jaccard similarity would predict an overlap of* $\frac{10}{27} = 0.37$ *under set semantics, an overlap of* $2 \cdot \frac{12}{42} = 0.57$ *under scaled bag semantics, and* $\frac{12}{18} = 0.67$ *under bag semantics normalized by the smaller table area. The true value of the overlap ratio between the tables is* $\frac{6}{18} = 0.33$*, and is overestimated by all methods.*